

C#.NET



Mr. Bangar Raju

NARESHO
technologies

Differences between String and String Builder

Understanding the Differences Between String and StringBuilder in C#.NET

By Albert Harmon Published September 22, 2016 ⌚ 3 min read [Technology](#)



TLDR This blog post explores the key differences between String and StringBuilder in C#.NET, highlighting their mutability, memory allocation, performance implications, and when to use each for string manipulation.

In the world of C#.NET programming, understanding the differences between String and StringBuilder is crucial, especially for entry-level programmers. This knowledge not only aids in writing efficient code but also prepares candidates for technical interviews. In this post, we will delve into the characteristics of both String and StringBuilder, their mutability, memory allocation, and performance considerations.

Mutability of Strings

Strings in C#.NET are immutable. This means that once a string is created, it cannot be modified. Any operation that appears to modify a string actually creates a new string in memory. For example, when you concatenate a string with another value, a new string is created rather than modifying the original string.

Example of String Immutability

Consider the following code:

```
string str = "Hello";  
str = str + " World";
```

In this example, the original string "Hello" remains unchanged. Instead, a new string "Hello World" is created, and the variable `str` now points to this new string. This behavior leads to multiple copies of

strings being stored in memory, which can be inefficient if many modifications are made.

Memory Allocation for Strings

When a string is declared, memory is allocated on the heap for that string. Each time a modification is made, a new memory allocation occurs. This can lead to significant memory overhead if many modifications are performed on strings.

Memory Allocation Example

If you perform multiple concatenations:

```
string str = "Hello";  
str = str + " World";  
str = str + "!";
```

Each concatenation results in a new string being created in memory, leading to three separate copies:

- "Hello"
- "Hello World"
- "Hello World!"

StringBuilder, on the other hand, is mutable. This means that you can modify the contents of a StringBuilder instance without creating new copies in memory. This is particularly useful when you need to perform numerous modifications to a string.

Creating a StringBuilder

You can create a StringBuilder as follows:

```
StringBuilder sb = new StringBuilder("Hello");
```

When you append to a StringBuilder, it modifies the existing instance rather than creating a new one:

```
sb.Append(" World");
```

Performance Considerations

The performance difference between String and StringBuilder becomes evident when dealing with a large number of modifications. Using String for frequent modifications can lead to performance overhead due to the creation of multiple copies.

Performance Example

To illustrate this, consider the following loop that concatenates strings:

```
string str = "";
for (int i = 0; i < 100000; i++)
{
    str += "Hello";
}
```

This operation can take a significant amount of time due to the creation of 100,000 string copies. In contrast, using `StringBuilder`:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++)
{
    sb.Append("Hello");
}
```

This operation is much faster because it modifies the existing `StringBuilder` instance without creating new copies.

Conclusion

In summary, the choice between String and StringBuilder in C#.NET depends on the specific use case. For scenarios involving frequent modifications, StringBuilder is the preferred choice due to its mutability and efficiency in memory usage. Conversely, if you are working with static or infrequently modified strings, using String is perfectly acceptable. Understanding these differences will help you write more efficient and effective C#.NET code.